

Московский Физико-Технический Институт

Математический анализ

Конспекты лекций

Студент: Антон Пумпеликсов

Преподаватель: Тюлень

Осень 2025

Содержание

Лекция 1

Введение в математический анализ

1 сентября 2025

1. Вещественные числа

Множество вещественных чисел $\mathbb{R}$ является фундаментальным объектом математического анализа. Оно обладает структурой **полного упорядоченного поля**.

1.1. Аксиомы поля

Для любых $a, b, c \in \mathbb{R}$ выполняются следующие свойства:

1. Коммутативность: $a + b = b + a$, $ab = ba$
2. Ассоциативность: $(a + b) + c = a + (b + c)$
3. Дистрибутивность: $a(b + c) = ab + ac$
4. Существование нейтральных элементов: 0 и 1
5. Существование обратных элементов

1.2. Аксиома полноты

Определение 1.1 (Супремум). Пусть $A \subset \mathbb{R}$ — непустое ограниченное сверху множество. **Супремумом** множества A называется наименьшая верхняя грань:

$$\sup A = \min \{M \in \mathbb{R} : \forall a \in A, a \leq M\}$$

Теорема 1.1 (О супремуме). *Всякое непустое ограниченное сверху множество $A \subset \mathbb{R}$ имеет супремум.*

Доказательство. Это свойство принимается как аксиома полноты вещественных чисел и не может быть доказано из остальных аксиом. $\square$

2. Последовательности

Определение 2.1 (Последовательность). **Последовательность** — это функция $a : \mathbb{N} \rightarrow \mathbb{R}$, обозначаемая $(a_n)_{n=1}^{\infty}$ или просто (a_n) .

2.1. Сходимость последовательностей

Определение 2.2 (Предел последовательности). Число $L \in \mathbb{R}$ называется **пределом** последовательности (a_n) , если:

$$\forall \varepsilon > 0 \quad \exists N \in \mathbb{N} : \forall n > N \Rightarrow |a_n - L| < \varepsilon$$

Обозначение: $\lim_{n \rightarrow \infty} a_n = L$ или $a_n \rightarrow L$.

Теорема 2.1 (Единственность предела). Если последовательность (a_n) сходится, то её предел единственен.

Доказательство. Предположим, что $a_n \rightarrow L_1$ и $a_n \rightarrow L_2$, где $L_1 \neq L_2$. Положим $\varepsilon = \frac{|L_1 - L_2|}{2} > 0$.

По определению предела:

- $\exists N_1 : \forall n > N_1 \Rightarrow |a_n - L_1| < \varepsilon$
- $\exists N_2 : \forall n > N_2 \Rightarrow |a_n - L_2| < \varepsilon$

При $n > \max(N_1, N_2)$:

$$|L_1 - L_2| \leq |a_n - L_1| + |a_n - L_2| < 2\varepsilon = |L_1 - L_2|$$

Противоречие. Следовательно, $L_1 = L_2$. □

3. Примеры

Пример 3.1. Докажем, что $\lim_{n \rightarrow \infty} \frac{1}{n} = 0$.

Пусть $\varepsilon > 0$. Выберем $N = \lceil \frac{1}{\varepsilon} \rceil$. Тогда для всех $n > N$:

$$\left| \frac{1}{n} - 0 \right| = \frac{1}{n} < \frac{1}{N} \leq \varepsilon$$

Важно: Понимание определения предела через ε - N — ключевой навык в математическом анализе.

Замечание. На практике часто используют арифметику пределов вместо прямой проверки определения.

Лекция 2

Примеры кода на разных языках

8 сентября 2025

Эта лекция демонстрирует возможности подсветки синтаксиса для различных языков программирования.

1. Python

Python — высокоуровневый язык программирования с динамической типизацией.

```
1 from typing import List, Optional
2 from dataclasses import dataclass
3
4 @dataclass
5 class Point:
6     x: float
7     y: float
8
9     def distance(self, other: "Point") -> float:
10         """Вычисляет расстояние до другой точки"""
11         return ((self.x - other.x)**2 + (self.y - other.y)**2) ** 0.5
12
13 def find_closest(points: List[Point], target: Point) -> Optional[Point]:
14     if not points:
15         return None
16     return min(points, key=lambda p: p.distance(target))
17
18 # Пример использования
19 points = [Point(0, 0), Point(3, 4), Point(1, 1)]
20 target = Point(2, 2)
21 closest = find_closest(points, target)
22 print(f"Ближайшая точка: {closest}") # Point(x=1, y=1)
```

2. C++

Современный C++ (C++17/20) с использованием STL и умных указателей.

```
1 #include <iostream>
2 #include <vector>
3 #include <memory>
4 #include <algorithm>
5
6 template<typename T>
7 class Stack {
8 private:
9     std::vector<T> data;
10
11 public:
12     void push(T value) {
13         data.push_back(std::move(value));
14     }
15
16     std::optional<T> pop() {
17         if (data.empty()) return std::nullopt;
18         T value = std::move(data.back());
19         data.pop_back();
20         return value;
21     }
22
23     [[nodiscard]] bool empty() const noexcept {
24         return data.empty();
25     }
26 };
27
28 int main() {
29     auto stack = std::make_unique<Stack<int>>();
30     stack->push(42);
31
32     if (auto val = stack->pop()) {
33         std::cout << "Popped: " << *val << std::endl;
34     }
35     return 0;
36 }
```

3. Rust

Rust — язык системного программирования с гарантиями безопасности памяти.

```
1 use std::collections::HashMap;
```

```
2
3 #[derive(Debug, Clone)]
4 struct User {
5     name: String,
6     age: u32,
7 }
8
9 impl User {
10     fn new(name: impl Into<String>, age: u32) -> Self {
11         Self { name: name.into(), age }
12     }
13 }
14
15 fn group_by_age(users: &[User]) -> HashMap<u32, Vec<&User>> {
16     let mut groups: HashMap<u32, Vec<&User>> = HashMap::new();
17
18     for user in users {
19         groups.entry(user.age).or_default().push(user);
20     }
21
22     groups
23 }
24
25 fn main() {
26     let users = vec![
27         User::new("Alice", 25),
28         User::new("Bob", 30),
29         User::new("Charlie", 25),
30     ];
31
32     let grouped = group_by_age(&users);
33     println!("{:#?}", grouped);
34 }
```

4. Go

Go — компилируемый язык от Google с встроенной поддержкой конкурентности.

```
1 package main
2
3 import (
4     "fmt"
5     "sync"
```

```
6 )
7
8 type Counter struct {
9     mu    sync.Mutex
10    value int
11 }
12
13 func (c *Counter) Increment() {
14     c.mu.Lock()
15     defer c.mu.Unlock()
16     c.value++
17 }
18
19 func (c *Counter) Value() int {
20     c.mu.Lock()
21     defer c.mu.Unlock()
22     return c.value
23 }
24
25 func main() {
26     counter := &Counter{}
27     var wg sync.WaitGroup
28
29     for i := 0; i < 1000; i++ {
30         wg.Add(1)
31         go func() {
32             defer wg.Done()
33             counter.Increment()
34         }()
35     }
36
37     wg.Wait()
38     fmt.Printf("Final value: %d\n", counter.Value())
39 }
```

5. JavaScript/TypeScript

Пример асинхронного кода на JavaScript.

```
1 interface User {
2     id: number;
3     name: string;
4     email: string;
```



```
5 }
6
7 async function fetchUsers(): Promise<User[]> {
8     const response = await fetch('/api/users');
9     if (!response.ok) {
10         throw new Error(`HTTP error: ${response.status}`);
11     }
12     return response.json();
13 }
14
15 const processUsers = async () => {
16     try {
17         const users = await fetchUsers();
18         const names = users.map(u => u.name);
19         console.log('Users:', names.join(', '));
20     } catch (error) {
21         console.error('Failed to fetch users:', error);
22     }
23 };
24
25 // Использование Promise.all для параллельных запросов
26 const [users, posts] = await Promise.all([
27     fetchUsers(),
28     fetchPosts()
29 ]);
```

6. SQL

Пример сложного SQL-запроса с CTE и оконными функциями.

```
1 WITH monthly_sales AS (
2     SELECT
3         DATE_TRUNC('month', order_date) AS month,
4         product_id,
5         SUM(quantity * price) AS revenue
6     FROM orders
7     WHERE order_date >= '2024-01-01'
8     GROUP BY 1, 2
9 ),
10 ranked_products AS (
11     SELECT
12         month,
13         product_id,
```

```
14     revenue,
15     ROW_NUMBER() OVER (
16         PARTITION BY month
17         ORDER BY revenue DESC
18     ) AS rank
19 FROM monthly_sales
20 )
21 SELECT
22     r.month,
23     p.name AS product_name,
24     r.revenue,
25     r.rank
26 FROM ranked_products r
27 JOIN products p ON r.product_id = p.id
28 WHERE r.rank <= 5
29 ORDER BY r.month, r.rank;
```

7. Haskell

Функциональное программирование на Haskell.

```
1 module Main where
2
3 import Data.List (sortBy)
4 import Data.Ord (comparing)
5
6 -- Определениетипа данных
7 data Tree a = Empty | Node a (Tree a) (Tree a)
8     deriving (Show, Eq)
9
10 -- Вставка в бинарное дерево поиска
11 insert :: Ord a => a -> Tree a -> Tree a
12 insert x Empty = Node x Empty Empty
13 insert x (Node y left right)
14     | x < y      = Node y (insert x left) right
15     | otherwise = Node y left (insert x right)
16
17 -- Обход дерева (in-order)
18 inorder :: Tree a -> [a]
19 inorder Empty = []
20 inorder (Node x left right) = inorder left ++ [x] ++ inorder right
21
22 -- Создание дерева из списка
```

```
23 fromList :: Ord a => [a] -> Tree a
24 fromList = foldr insert Empty
25
26 main :: IO ()
27 main = do
28     let tree = fromList [5, 3, 7, 1, 4, 6, 8]
29     print $ inorder tree -- [1,3,4,5,6,7,8]
```

8. Bash

Пример shell-скрипта для автоматизации.

```
1 #!/bin/bash
2 set -euo pipefail
3
4 # Функция для логирования
5 log() {
6     echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"
7 }
8
9 # Проверка зависимостей
10 check_deps() {
11     local deps=("git" "docker" "jq")
12     for dep in "${deps[@]}; do
13         if ! command -v "$dep" &> /dev/null; then
14             log "ERROR: $dep is not installed"
15             exit 1
16         fi
17     done
18 }
19
20 # Основная логика
21 main() {
22     check_deps
23
24     log "Starting deployment..."
25
26     # Получение последней версии
27     VERSION=$(git describe --tags --always)
28     log "Version: $VERSION"
29
30     # Сборка и деплой
31     docker build -t "myapp:$VERSION" .
```

```
32     docker push "myapp:$VERSION"
33
34     log "Deployment complete!"
35 }
36
37 main "$@"
```